

信念集合で条件付けた連続行動拡散方策と DPPO 型 PPO 目的

Belief-Score-Based-Model 実験仕様

August 12, 2026

Abstract

本稿は、スコアで表した粒子信念を条件として連続行動を生成する DDPM 方策と、その各逆拡散遷移へ PPO を適用する学習目的を定義する。最終行動の周辺尤度は一般に計算不能であるため、DPPO[1] と同様に逆拡散過程を内側の MDP として展開し、各 Gaussian 遷移の尤度比を個別に clip する。一方、本実験はデモンストレーションによる事前学習を用いない from-scratch 設定であり、標準 DPPO の主設定とは異なる。信念 energy は、rollout 時に保存した Gaussian noise から現在 parameters で ULA と belief recurrence を再 parameterize して end-to-end に更新する。本稿では episode 全体へ逆伝播する full と、環境時刻境界だけを切る tbptt_1 を区別して実装対象を明示する。active production では、score-belief Transformer と GRU の双方へ同一の DiffusionPolicy と PPO 目的を接続し、action family を揃えて表現・記憶機構だけを比較する。

1 記法と信念条件

環境時刻を t 、観測を o_t 、環境報酬を r_t 、連続行動を $a_t \in \mathcal{A} \subset \mathbb{R}^{d_a}$ とする。信念は K_b 個の粒子と、その位置で評価した score の集合

$$B_t = \{(s_{t,i}, q_{t,i})\}_{i=1}^{K_b}, \quad q_{t,i} = \mathfrak{s}_t(s_{t,i}) \in \mathbb{R}^{d_s} \quad (1)$$

で近似する。粒子順序に依存しない Set Transformer encoder と pooling を用いて

$$c_t = E_\phi(B_t) = \text{Pool}\left(\text{SetTransformer}_\phi([s_{t,i}, q_{t,i}]_{i=1}^{K_b})\right) \in \mathbb{R}^{d_c} \quad (2)$$

を作る。critic と拡散方策はともに c_t で条件付ける。単一行動を生成する場合を記述するが、 a_t を $H_a \times d_a$ 次元の action chunk に置き換えても数式は同じである。

active 比較の GRU では、部分観測と正規化した直前行動を入力して

$$h_t = \text{GRU}_\omega([o_t, a_{t-1}], h_{t-1}), \quad c_t = W_c h_t \quad (3)$$

を作る。式 (2) と式 (3) の後段には、同一構成の critic と DiffusionPolicy を接続する。reverse step 数、noise schedule、variance floor、denoising advantage 重み、per-step PPO clipping も共通である。したがって連続 task の active 比較は diffusion 対 Gaussian ではなく、score-belief 表現対 GRU 記憶の比較である。離散 CartPole では両者に同じ categorical head を使う。

2 信念条件付き DDPM 方策

2.1 forward process と reverse process

拡散 step 数を N とし、 $\beta_n \in (0, 1)$ 、 $\alpha_n = 1 - \beta_n$ 、 $\bar{\alpha}_n = \prod_{j=1}^n \alpha_j$ と置く。正規化済み行動 x^0 に対する forward process は

$$q(x^n | x^{n-1}) = \mathcal{N}(\sqrt{\alpha_n} x^{n-1}, \beta_n I), \quad (4)$$

$$q(x^n | x^0) = \mathcal{N}(\sqrt{\bar{\alpha}_n} x^0, (1 - \bar{\alpha}_n) I) \quad (5)$$

である。方策として使う reverse chain は

$$x_t^N \sim \mathcal{N}(0, I), \quad p_\theta(x_t^{n-1} | x_t^n, c_t) = \mathcal{N}(\mu_\theta(x_t^n, n, c_t), \sigma_n^2 I), \quad n = N, \dots, 1, \quad (6)$$

とする。noise-prediction parameterization では

$$\mu_\theta(x^n, n, c) = \frac{1}{\sqrt{\alpha_n}} \left(x^n - \frac{\beta_n}{\sqrt{1 - \bar{\alpha}_n}} \epsilon_\theta(x^n, n, c) \right). \quad (7)$$

実装では posterior variance に下限 $\sigma_{\min}^2 > 0$ を設ける。これは最終 step を含む各遷移の Gaussian log-density を有限に保ち、学習中の探索を確保するためである。

環境へ渡す有界行動は、既知の決定論的変換

$$a_t = T(x_t^0) = \tanh(x_t^0) \odot \frac{a_{\max} - a_{\min}}{2} + \frac{a_{\max} + a_{\min}}{2} \quad (8)$$

で作る。PPO で評価する確率変数は、変換後の周辺分布ではなく式 (6) の各内部遷移である。

2.2 拡散過程を内側 MDP として扱う

環境時刻 t ごとに、 $n = N, N-1, \dots, 1$ の N 個の内側遷移を置く。内側状態、内側行動、報酬をそれぞれ

$$\bar{s}_{t,n} = (c_t, x_t^n), \quad \bar{a}_{t,n} = x_t^{n-1}, \quad \bar{r}_{t,n} = \begin{cases} 0, & n > 1, \\ r_t, & n = 1 \end{cases} \quad (9)$$

とする。したがって各内側方策 $p_\theta(x_t^{n-1} | x_t^n, c_t)$ は明示的な Gaussian likelihood を持つ。最終行動の周辺密度

$$\pi_\theta(x_t^0 | c_t) = \int p(x_t^N) \prod_{n=1}^N p_\theta(x_t^{n-1} | x_t^n, c_t) dx_t^{1:N} \quad (10)$$

は計算せず、trajectory 上で観測された各遷移を PPO sample として使う。

3 DPPO 型 clipped PPO 目的

3.1 各 denoising transition の尤度比

rollout 時の parameters を $(\theta_{\text{old}}, \phi_{\text{old}})$ とする。保存した reverse chain (x_t^n, x_t^{n-1}) と old log-density、および式 (26) で current parameters から再構成する $c_t(\phi)$ に対し、各 step の log ratio を

$$\ell_{t,n}(\theta, \phi) = \log p_\theta(x_t^{n-1} | x_t^n, c_t(\phi)), \quad (11)$$

$$\rho_{t,n}(\theta, \phi) = \exp(\ell_{t,n}(\theta, \phi) - \ell_{t,n}(\theta_{\text{old}}, \phi_{\text{old}})) \quad (12)$$

と定める。重要なのは、**各 step の ratio を個別に clip すること**である。式 (10) の ratio も、 $\prod_n \rho_{t,n}$ も PPO 目的には使用しない。

3.2 環境 GAE、critic、denoising advantage

critic は内側の noisy action には依存せず、環境時刻ごとに

$$V_\psi(c_t) \simeq \mathbb{E} \left[\sum_{j \geq 0} \gamma_{\text{env}}^j r_{t+j} \mid c_t \right] \quad (13)$$

を推定する。episode 終了 indicator を $d_t \in \{0, 1\}$ として

$$\delta_t = r_t + \gamma_{\text{env}}(1 - d_t)V_{\psi_{\text{old}}}(c_{t+1}) - V_{\psi_{\text{old}}}(c_t), \quad (14)$$

$$\hat{A}_t^{\text{env}} = \sum_{j=0}^{T-t-1} (\gamma_{\text{env}}\lambda)^j \left(\prod_{m=0}^{j-1} (1 - d_{t+m}) \right) \delta_{t+j}, \quad (15)$$

$$\hat{R}_t = \hat{A}_t^{\text{env}} + V_{\psi_{\text{old}}}(c_t) \quad (16)$$

とする。内側 step には同じ環境 advantage を複製し、

$$\hat{A}_{t,n}^{\text{den}} = \gamma_{\text{den}}^{n-1} \hat{A}_t^{\text{env}}, \quad 0 < \gamma_{\text{den}} \leq 1 \quad (17)$$

と重み付けする。 $n = N$ が最初で最も noisy な遷移なので、 $\gamma_{\text{den}} < 1$ では最初の step ほど強く downweight され、 $n = 1$ の最終 step の重みは 1 になる。

PPO の surrogate と value loss は

$$L_{\text{clip}}(\theta, \phi) = \frac{1}{TN} \sum_{t,n} \min \left\{ \rho_{t,n} \hat{A}_{t,n}^{\text{den}}, \text{clip}(\rho_{t,n}, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_{t,n}^{\text{den}} \right\}, \quad (18)$$

$$L_V(\psi, \phi) = \frac{1}{2T} \sum_t \left(V_{\psi}(c_t(\phi)) - \hat{R}_t \right)^2, \quad (19)$$

$$L_{\text{total}} = -L_{\text{clip}} + c_V L_V - c_H \mathcal{H} \quad (20)$$

である。 L_{clip} が方策更新項、 L_V が critic の value regression、 $\mathcal{H}$ が方策 entropy である。これらは標準的な PPO の三要素であるが、本比較実験では debug・production とともに $c_H = 0$ とし、entropy bonus を最適化項から除外する。active な連続比較は両手法とも固定 schedule の DiffusionPolicy であり、最適化する目的は $-L_{\text{clip}} + c_V L_V$ で共通である。実装では PPO の clipped objective と gradient norm clipping を用いる。approximate KL と clip fraction は診断値として記録するだけで、更新の打ち切りや minibatch の拒否には使用しない。また log ratio に独自の hard clamp を置かず、式 (12) と PPO clip をそのまま用いる。標準 ULA の drift や状態を変える score/particle clamp も main 設定では 0 (無効) とする。pilot で不安定性が出た場合は、まず学習率と ULA step size を明示的に再設定する。

4 再 parameterize した belief recurrence の全微分

4.1 明示 noise による再生

信念 energy の parameters を $\eta = (\eta_0, \eta_f, \eta_u)$ とする。初期時刻専用の energy network は通常時の観測・遷移 energy とは parameters を共有せず、実装する potential は

$$E_{\eta}(s; o_t, B_{t-1}, a_{t-1}) = \begin{cases} g_{\eta_0}(o_0, s) - \frac{1}{2}\|s\|^2, & \text{episode 初期時刻,} \\ f_{\eta_f}(o_t, s, a_{t-1}) + \log \text{meanexp}_{s^- \in B_{t-1}} u_{\eta_u}(s, s^-, a_{t-1}), & \text{それ以外} \end{cases} \quad (21)$$

とする。 $-\|s\|^2/2$ は parameter regularization ではなく、初期密度の裾と latent scale を固定する標準 Gaussian 基準密度の log density である。無制限の関数族ならこの項を g_{η_0} へ吸収する再 parameterization は可能だが、有限幅 MLP が非有界な latent 空間全体で同じ二次 tail を正確に表す保証はない。そのため実装では normalizable な裾と latent scale を明示する基準項として分離する。rollout 時には粒子座標を PPO 更新用の固定入力として保存するのではなく、各環境時刻 t について ULA 初期 noise $z_{t,i} \sim \mathcal{N}(0, I)$ と各 step の noise $\epsilon_{t,l,i} \sim \mathcal{N}(0, I)$ を保存する。PPO 更新時には観

測列、実現済み行動列、episode-start mask、およびこれらの noise を固定し、現在の η から

$$r_{t,0,i}^\eta = z_{t,i}, \quad (22)$$

$$g_{t,l,i}^\eta = \nabla_s E_\eta \left(r_{t,l-1,i}^\eta; o_t, B_{t-1}^\eta, a_{t-1} \right), \quad (23)$$

$$r_{t,l,i}^\eta = r_{t,l-1,i}^\eta + \alpha_l g_{t,l,i}^\eta + \sqrt{2\alpha_l} \epsilon_{t,l,i}, \quad l = 1, \dots, L, \quad (24)$$

$$s_{t,i}^\eta = r_{t,L,i}^\eta, \quad q_{t,i}^\eta = \nabla_s E_\eta \left(s_{t,i}^\eta; o_t, B_{t-1}^\eta, a_{t-1} \right), \quad (25)$$

$$B_t^\eta = \{(s_{t,i}^\eta, q_{t,i}^\eta)\}_{i=1}^{K_b}, \quad c_t^\eta = E_\phi(B_t^\eta) \quad (26)$$

を順に再計算する。全 ULA step と最後の score 計算で `create_graph=True` を使うため、PPO/value loss から粒子生成経路、初期 energy、観測 energy、遷移 energy へ pathwise gradient が流れる。noise の分布は parameters に依存しないので、保存 noise 自体は微分しない。

PPO が固定 data として扱うのは o_t 、実現済み a_{t-1} 、 r_t 、終了 mask、old log-prob、advantage と return である。環境状態遷移や観測生成を再現するモデルは使わず、環境へは微分しない。この意味で学習は model-free でありながら、agent 内部の belief recurrence には全勾配を流す。

4.2 full と tbptt_1

`full` では、rollout 開始時に episode が継続中なら、その episode の開始から rollout 境界までの観測、実現済み行動、noise からなる prefix も保持する。各 PPO minibatch で episode-start から対象 loss 時刻まで式 (26) を現在 parameters で再生し、真の episode reset だけで graph を切る。したがって random に flatten した環境時刻を独立 sample として扱ってはならず、時間順の sequence minibatch を用いる。

`tbptt_1` は同じ forward 再生を用いるが、各環境時刻の冒頭だけ

$$B_{t-1}^{\text{in}} = \text{stopgrad}(B_{t-1}^\eta) \quad (27)$$

とする。現在時刻内の L 個の ULA step と最後の score 計算は detach しない。よって両 mode は同じ noise と parameters に対して同一の粒子・action distribution を返し、違いは時間方向の gradient 範囲だけである。

optimizer step の後には、それまでの未終了 episode prefix を更新後 parameters で再生し、次 rollout の belief context を再構築する。これにより、更新前 parameters で作った stale particle を次 rollout へ持ち越さない。PPO の複数 epoch でも計算 graph を使い回さず、各 optimizer step の current parameters で sequence を再生する。

`full` の graph memory は episode 長に比例するため、実装では non-reentrant activation checkpoint `torch.utils.checkpoint(..., use_reentrant=False)` を用いる。backward 時に forward activation を再計算するだけなので、detach や no-grad による近似と異なり、数学的な勾配は変えない。計算量は増えるため wall-clock と peak GPU memory を mode 別に記録する。さらに approximate KL、clip fraction、global gradient norm に加え、初期 energy g_0 、観測 energy f 、遷移 energy u それぞれの pre-clip gradient norm を監視する。初期時刻と再帰時刻を分けた score/particle の L2 norm、各区分の particle 数、score/particle の non-finite 数も記録する。これらは診断値であり、閾値による optimizer step の拒否には使わない。

production では論理 PPO minibatch を $512 = 4 \times 128$ 遷移のまま保ち、score belief の sequence 再生だけを、時間方向を切らずに 128 遷移ずつ 4 回処理する。各 memory microbatch の平均 loss を 1/4 倍して backward し、4 回分の gradient を加算した後に gradient clipping と optimizer step をそれぞれ 1 回だけ行う。したがって、浮動小数点の加算順による微小差を除けば 512 遷移の平均 loss に対する更新と同じであり、`full` と `tbptt_1` の時間方向の gradient 境界も変えない。

GRU にも同じ時間範囲の二条件を実装する。`full` では、rollout 開始時に episode が継続中なら観測・直前行動からなる episode prefix を保持し、current parameters で episode 開始から式 (3) を再生する。rollout 境界では graph を切らず、真の episode reset だけを境界とする。`tbptt_1` では同じ入力列と parameters を用いるが、各環境時刻の冒頭で

$$h_{t-1}^{\text{in}} = \text{stopgrad}(h_{t-1}) \quad (28)$$

とする。両 mode の hidden、方策分布、value という forward 値は同一であり、差は時間方向の gradient 範囲だけである。GRU の PPO minibatch は固定長 time chunk へ切らず、完全な時間軸を保ったまま環境軸だけを分割する。optimizer step 後は未終了 episode prefix を更新後 parameters で再生し、次 rollout 用の online hidden を再構築する。

5 学習アルゴリズム

アルゴリズム 1 Belief-conditioned DDPM policy の DPPO 型 from-scratch 学習

入力: 並列環境、rollout 長 T 、粒子数 K_b 、ULA step 数 L 、拡散 step 数 N 、gradient mode

出力: 更新済み信念 energy、set encoder、denoiser、critic

```

1: 全 network をランダム初期化し、episode 初期粒子を設定する
2: for 各 PPO iteration do
3:   rollout 用 parameters を old policy として固定する
4:   for  $t = 0, \dots, T - 1$  do
5:      $z_{t,1:K_b}, \epsilon_{t,1:L,1:K_b}$  を生成・保存して ULA を  $L$  step 実行する
6:      $q_{t,i} = \nabla_s E_{\eta_{\text{old}}}(s; \cdot) |_{s=\bar{s}_{t,i}}$  と  $c_t$  を計算する
7:      $x_t^N \sim \mathcal{N}(0, I)$  を生成する
8:     for  $n = N, N - 1, \dots, 1$  do
9:        $x_t^{n-1} \sim p_{\theta_{\text{old}}}(\cdot | x_t^n, c_t)$ 
10:       $(x_t^n, x_t^{n-1}, \ell_{t,n}^{\text{old}})$  を保存する
11:    end for
12:     $a_t = T(x_t^0)$  を環境へ渡し、 $o_t, a_t, r_t, d_t, V_{\psi_{\text{old}}}(c_t)$  を保存する
13:  end for
14:  式 (16) で  $\hat{A}_t^{\text{env}}, \hat{R}_t$  を計算し、advantage を正規化する
15:  for 各 PPO epoch と論理 sequence minibatch do
16:    gradient を初期化する
17:    for 4 個の memory microbatch (各 128 遷移) do
18:      episode-start から保存 noise で式 (26) を current parameters で再生する
19:      if gradient mode が tbptt_1 then
20:        各環境時刻の入力 belief だけ式 (27) で detach する
21:      end if
22:      全保存遷移の new log-prob と式 (12) の  $\rho_{t,n}$  を計算する
23:      式 (20) の loss を 1/4 倍し、checkpoint 付き backpropagation で加算する
24:    end for
25:    加算 gradient を 1 回 clip し、optimizer step を 1 回行う
26:  end for
27:  未終了 episode prefix を current parameters で再生し、次 rollout の context を再構築する
28: end for
```

6 行動選択：argmax を使わない

式 (10) の密度を直接評価できないため、通常の拡散方策では

$$\arg \max_a \pi_{\theta}(a | B_t) \quad (29)$$

を計算できない。学習中は必ず逆拡散 chain を sample して $a_t \sim \pi_{\theta}(\cdot | B_t)$ とする。
評価時の再現可能な決定論的規約として、本実装は

$$x_t^N = 0, \quad x_t^{n-1} = \mu_{\theta}(x_t^n, n, c_t), \quad a_t = T(x_t^0) \quad (30)$$

という **mean-chain** を使う。これは各 conditional Gaussian の平均を順に通す規約であり、周辺分布の平均や MAP と等しいとは限らない。報告時には stochastic evaluation と mean-chain evaluation を区別する。さらに score 系では、mean-chain にしても belief の初期粒子と ULA noise は sample される。したがって「決定論的」は action readout だけを指し、agent 全体の軌跡が確定するという意

味ではない。`mean_chain` は return 等の性能指標ではなく、この action readout 規約の名前である。tanh Gaussian を使う非 active ablation の `mean_action` も同様に、pre-tanh Gaussian 平均を tanh と action scale へ通す readout 名であり、metric ではない。active な連続比較は score Transformer と GRU の双方が `mean_chain` を使うため、deterministic readout も揃う。それでも事前に一つの primary metric を選ぶなら、探索を含む学習方策そのものを評価するため、held-out episode における stochastic evaluation の最終 return を training seed 間で集約した値を用いる。

7 標準 DPPO との差と limitations

7.1 from-scratch 設定

Ren らの DPPO[1] の中心設定は、デモンストレーションで behavior cloning した Diffusion Policy を RL で fine-tune するものである。早期 denoising step を凍結し、最後の $N' < N$ step だけ更新する設計も事前学習を前提とする。本実験では、信念 energy、encoder、denoiser、critic をすべてランダム初期化し、全 denoising step を環境報酬だけから学習する。DPPO の付録では from-scratch 実験もあるが、Gaussian PPO より学習が遅く、10-step diffusion で wall-clock が約 6 倍と報告されている。したがって本手法は「DPPO そのもの」ではなく、**DPPO 型の per-denoising-step PPO を from scratch で用いる手法**と記述する。

7.2 主な制約

1. **sample efficiency:** 環境 1 step を N 個の内側 step へ展開するため、Gaussian PPO より計算量が大きい。デモなしの複雑な task では探索が成立しない可能性がある。
2. **recurrent gradient の計算量:** full は episode-start から全 ULA step を再計算するため、時間、粒子数、ULA step 数に対して高コストである。tbptt_1 は時間方向を切る ablation であり、full-gradient の代替結果として混同せず mode 別に報告する。
3. **belief の非同定性:** PPO 報酬だけでは f_η, u_η が真の観測尤度・遷移密度になる保証はない。得られるものは制御に有用な task-oriented latent belief である。
4. **variance floor と有界化:** $\sigma_{\min}$ が小さすぎると ratio が不安定になり、大きすぎると最終行動が過度にランダムになる。tanh の飽和率も記録する必要がある。
5. **周辺尤度と MAP の欠如:** per-step PPO は拡張 MDP 上の surrogate である。最終 action の周辺 KL や MAP を直接制御しない。
6. **denoising index:** 保存 chain を $(x^N, x^{N-1}, \dots, x^0)$ の順に並べる場合、advantage 重みは $(\gamma_{\text{den}}^{N-1}, \dots, \gamma_{\text{den}}^0)$ の順でなければならない。逆順にすると noisy step を強調してしまう。
7. **比較の必要性:** 実装上は、同じ score Transformer を持つ `score_gaussian` に加え、observation MLP と oracle-state の tanh Gaussian PPO も同じ environment-step budget で比較できる。ただし active production は `score_transformer` と GRU に限定し、他の実装は local smoke で経路だけを確認する。active な連続比較では GRU にも提案法と同じ DiffusionPolicy、per-denoising-step PPO、schedule、advantage 重み、`mean_chain` readout を用いる。production の parameter 数も最大 0.116% 差へ合わせるが、各手法を個別に十分 tuning した比較ではない。
8. **action dimension:** Pendulum と MountainCar の 1 次元 Box に加え、5 次元 Light-Dark を比較する。ただし無限次元行動や多峰な action distribution に対する diffusion の利点までは検証しない。

8 実装と報告のチェック項目

拡散 chain N 、 β schedule、 $\sigma_{\min}$ 、 γ_{den} 、単一行動か chunk か。

PPO γ_{env} 、 λ 、clip 幅、epoch、論理 minibatch、memory microbatch、gradient accumulation、gradient clip、診断用 KL。

信念 粒子数、ULA step 数 $\cdot$ step size、particle/score clip、保存 noise、gradient mode。

評価 action readout の mean-chain/mean-action、共通指標の stochastic、episode 数、seed、action saturation 率。

計算量 環境 step 数に加えて mode 別 wall-clock、peak GPU memory、denoising transition 数。

比較 active な score Transformer/GRU の共通 action head と parameter 数、非 active な score Gaussian、observation MLP、oracle-state、Deep Sets。

診断 $g_0/f/u$ の pre-clip gradient norm、initial/recursive score $\cdot$ particle norm、sample 数、non-finite 数。

実装済み 6 手法の local smoke は、実行可能な 23 method/task 組と gradient mode 2 種類からなる 46 cells を 1 seed $\cdot$ 1 PPO update で確認する。一方、active production の比較行列は score_transformer と GRU の 2 手法 $\times$ 4 tasks $\times$ gradient mode 2 種類 $\times$ 5 seeds の合計 80 runs とし、集約表は mode と method/task の組ごとに 16 行を保持する。active production で score belief を持たない baseline は GRU だけである。GRU も両 mode label で実行し、full では episode prefix を rollout 境界越しに再生し、tbptt_1 では各環境 step の入力 hidden を detach する。したがって両 mode は同一 forward $\cdot$ 異なる temporal gradient 範囲を持つ、score belief と対応した GRU ablation である。

production の GRU は入力 encoder 幅を [348, 185]、hidden 幅を 176 とする。共通 policy/value head を含む parameter 数は次の通りで、最大絶対相対差は 0.116% である。

task	score Transformer	GRU	相対差
CartPole	347,462	347,060	-0.116%
Pendulum	348,357	348,759	+0.115%
MountainCarContinuous	348,229	348,411	+0.052%
Light-Dark ($N = 5$)	351,817	352,223	+0.115%

local 設定は経路確認用の小型 network であり、この capacity matching の対象外である。

References

- [1] Allen Z. Ren, Justin Lidard, Lars L. Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, Max Simchowitz, “Diffusion Policy Policy Optimization,” ICLR, 2025. https://proceedings.iclr.cc/paper_files/paper/2025/hash/c0749c39aaff9e9e4c91f7118bf21b1e-Abstract-Conference.html; <https://arxiv.org/abs/2409.00588>.
- [2] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov, “Proximal Policy Optimization Algorithms,” 2017. <https://arxiv.org/abs/1707.06347>.
- [3] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, Pieter Abbeel, “High-Dimensional Continuous Control Using Generalized Advantage Estimation,” ICLR, 2016. <https://arxiv.org/abs/1506.02438>.
- [4] Jonathan Ho, Ajay Jain, Pieter Abbeel, “Denoising Diffusion Probabilistic Models,” NeurIPS, 2020. <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>.
- [5] Ningyuan Yang, Jiaxuan Gao, Feng Gao, Yi Wu, Chao Yu, “Fine-tuning Diffusion Policies with Backpropagation Through Diffusion Timesteps,” 2025. <https://arxiv.org/abs/2505.10482>.